Course Code: 2165119 Course Name: ASD&D Lab

Class: TE Batch: T23 Semester: V

Student Name: Jainil Shah Roll No: 2301106

—

EXPERIMENT NO. 05

Aim: - To automate application deployment using GitHub and Jenkins.

Theory:

Continuous Integration and Continuous Deployment (CI/CD) pipelines automate the process of software delivery. Jenkins is a widely used open-source automation server that facilitates this by building, testing, and deploying code every time a change is made.

This experiment utilizes a Declarative Jenkins Pipeline, which defines pipeline, agent, stages, and steps as its core syntax elements. The standard automated pipeline consists of three main stages:

1. Build: This stage prepares and creates the application. For a compiled language, it compiles source code; for this experiment's HTML application, no complex compilation is required.
2. Test: This stage checks whether the application works correctly, running automated test scripts if configured.
3. Deploy: This final stage puts the successfully tested application into its target environment.

Webhooks play a crucial role in full automation. A webhook is an HTTP callback that occurs when something happens. The ultimate goal of this automated architecture is a seamless flow: a developer changes code, pushes it to GitHub, GitHub triggers a Webhook, the Webhook sends a payload to Jenkins, and Jenkins automatically executes the Build, Test, and Deploy stages to update the application without manual intervention.

IMPLEMENTATION

Step 1: Setting up the GitHub Repository and Application Code

A GitHub repository (ASDD_Exp5_Webhook) was created and cloned to the local environment. A sample index.html application file and a Jenkinsfile were created. The Jenkinsfile contained the declarative script defining the Build, Test, and Deploy stages.

```
PS D:\ASDD> git clone https://github.com/SwarajKanse/ASDD_Exp5_Webhook.git
PS D:\ASDD> cd ASDD_Exp5_Webhook
PS D:\ASDD\ASDD_Exp5_Webhook> Set-Content index.html "<h1>My Auto-Deployed App</h1>"
PS D:\ASDD\ASDD_Exp5_Webhook> notepad Jenkinsfile
PS D:\ASDD\ASDD_Exp5_Webhook> git add .
PS D:\ASDD\ASDD_Exp5_Webhook> git commit -m "Initial commit for Exp 5"
PS D:\ASDD\ASDD_Exp5_Webhook> git push origin main
```

Step 2: Configuring the Jenkins Pipeline

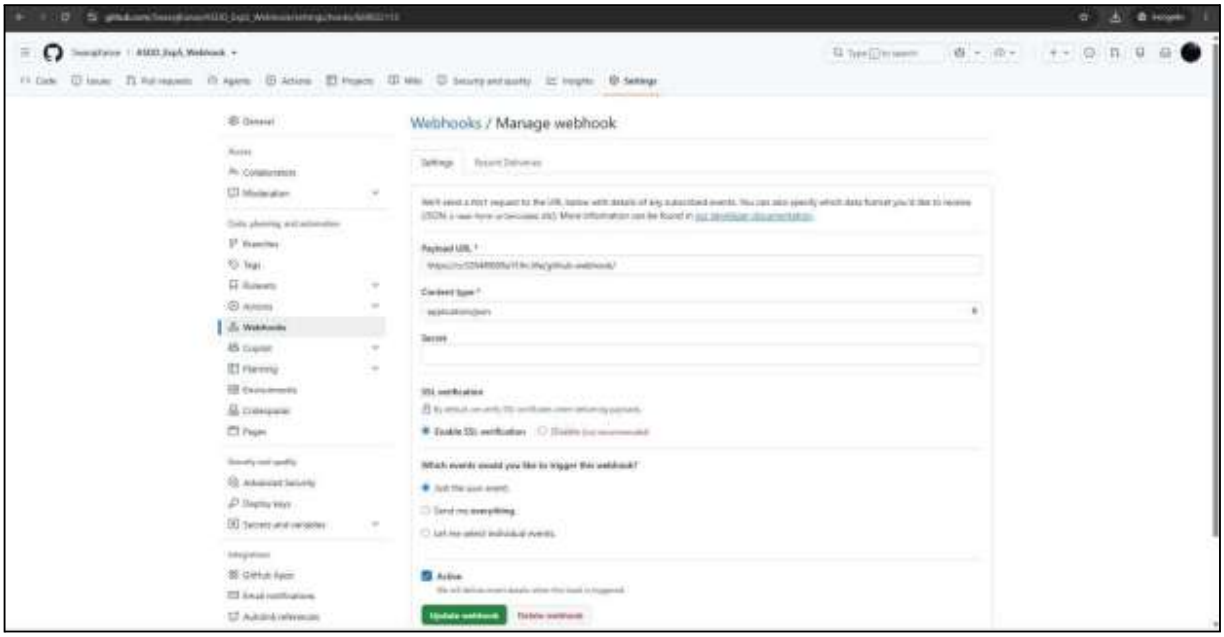
In the local Jenkins server (running on localhost:8081), a new Pipeline project named GitHub-Jenkins-Demo was created. Under the Build Triggers configuration, "GitHub hook trigger for GITScm polling" was enabled. The Pipeline definition was set to "Pipeline script from SCM", linked to the GitHub repository URL, and configured to look for the Jenkinsfile on the main branch.

Step 3: Creating a Public Tunnel and Configuring the GitHub Webhook

Because the local Jenkins server was running on a private lab network behind a firewall, GitHub's public servers could not reach it directly (resulting in a failed to connect to host error). To bypass this, an SSH tunnel (localhost.run) was used via PowerShell to generate a temporary public URL (https://cc3264f0608a1f.lhr.life) that forwarded traffic to localhost:8081.

```
PS C:\Users\Lab203> ssh -R 80:localhost:8081 nokey@localhost.run
```

This public URL was then added to the GitHub repository under Settings > Webhooks with the /github-webhook/ endpoint and application/json content type. The connection was established successfully.

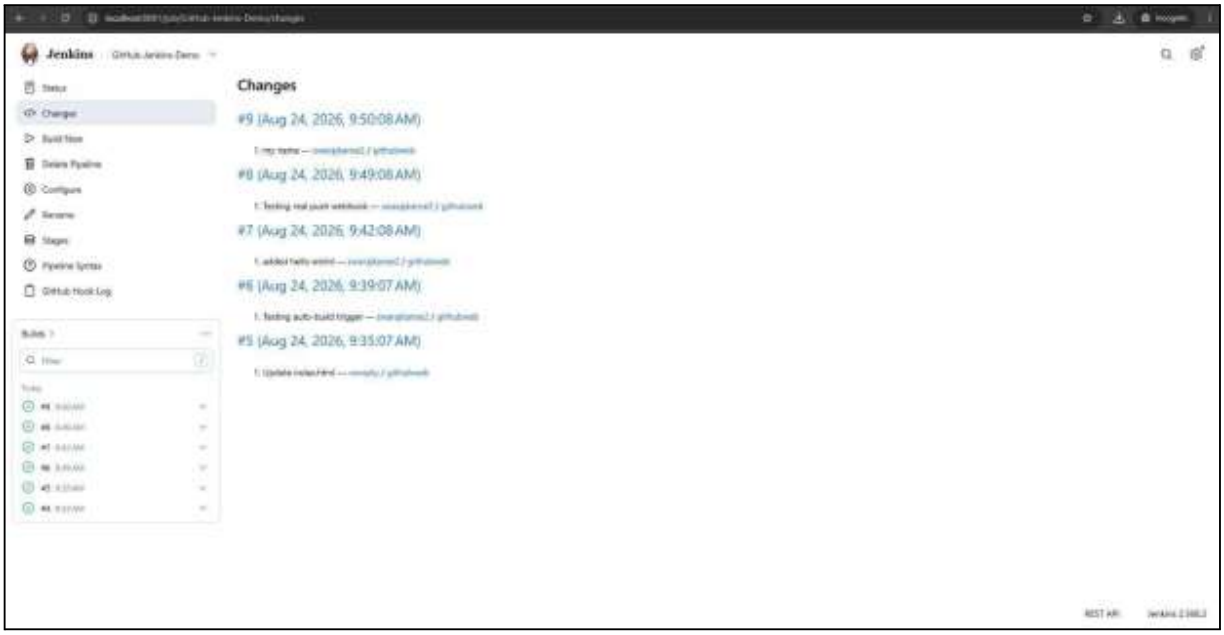


Step 4: Triggering and Verifying the Automated Build

To test the true automation (Push-to-Deploy), a modification was made to the local code and pushed to the remote repository.

```
PS D:\ASDD\ASDD_Exp5_Webhook> Set-Content index.html "<h1>True Webhook Push is Successful!</
PS D:\ASDD\ASDD_Exp5_Webhook> git add .
PS D:\ASDD\ASDD_Exp5_Webhook> git commit -m "Testing real push webhook"
PS D:\ASDD\ASDD_Exp5_Webhook> git push origin main
```

Immediately upon the push completing, GitHub fired the Webhook payload through the SSH tunnel to the Jenkins server. Observing the Jenkins dashboard confirmed that Build #8 ("Testing real push webhook") was triggered automatically without manual intervention, successfully executing the Build, Test, and Deploy stages.



Conclusion: Thus, we have successfully studied CI/CD pipelines and utilized Jenkins, GitHub, and Webhooks (with SSH tunneling) to create a fully automated application deployment workflow that triggers instantly upon code commits.